CargoOne

Technical Design Document

Proof-of-delivery capture, AI verification and Tally billing for Company A

GitHub	**https://github.com/priyadarshi7/CargoOne**
Live app	Not deployed. It runs locally and in CI; `deployment.md` has the steps.
Author	Priyadarshi (solo build)
Date	9 November 2026
Brief	Company A, Chhatrapati Sambhajinagar: POD capture, AI verification and Tally billing (Software Engineering core build)

1. Summary
2. The client and the problem
3. Scope: what I built and why
4. Architecture and why
5. Roles and access
6. Data model
7. The main flows
8. The AI part
9. AI benchmark
10. Integrations
11. GPS and tracking
12. The driver's side, and the money in it
13. UI and design decisions
14. Security and privacy (DPDP Act 2023)
15. Testing and quality
16. Sample data and how it is made
17. Assumptions
18. What I cut and what I would do next

Appendix A. Feature checklist

1. Summary

CargoOne takes the proof of delivery (POD) from a paper that arrives 10 to 20 days late to a photo that is checked, billed and visible on the day of delivery. The driver photographs the signed lorry receipt (LR) at the consignee's gate with one big button. The photo is compressed on the phone, stamped with the time and GPS position, and sent, even when the highway has no signal. An AI reads it while the driver is still there. Fixed rules, not the AI, then decide: verified, retake, or a person looks. Verified trips drop into a billing queue, become an invoice with the POD photo of every trip attached, and are pushed into Tally with one click.

The brief asks for a core slice (driver capture, AI check with retake, review queue, billing queue with export, missing-POD dashboard). That slice works end to end. On top of it the project also covers the whole shipment, from a client's request to the paid invoice, because a POD only matters in the context of the trip that produced it.

Brief asks for	Where it stands
Driver app: one big button, photo stamped with GPS and time	Built. Installable web app (PWA), icons, Hindi / Marathi / English voice prompts.
SMS upload link for hired drivers	Built. Private one-trip link, no install, no login. Twilio sends it; without credentials the link is shown on screen.
AI POD check, retake if needed	Built. Groq Qwen3.8 27B reads the photo, local Qwen3-VL + OCR is the fallback, fixed rules decide. Retakes capped at 2.
Billing queue by client and cycle, Tally export, POD images on invoices	Built. One-click push to Tally Prime, plus XML download. Invoice PDF carries every trip's POD photo.
Dashboard: trips without POD, POD-to-invoice days by branch, client POD link	Built. Computed live from trips; client link and client portal built.
Daily trip import with LR and e-way bill numbers	Built. CSV or Excel, checked first, written only after confirmation, safe to repeat.
Offline capture, images under 300 KB	Built. IndexedDB queue, in-order retry, idempotent upload, compression ladder.
DPDP Act 2023 for consignee data	Built. Notice, request register, erasure, 8-year retention job, audit trail.

> Reading guide: every number from the brief is used as given. Anything I had to assume is labelled **Assumption A1, A2, ...** and listed together in section 17.

2. The client and the problem

Company A, Chhatrapati Sambhajinagar: a full-truckload transporter for cement, steel and packaged goods firms. 140 own trucks plus about 200 hired, 210 staff, revenue 75 to 90 crore rupees a year. Drivers bring the signed LR or POD back 10 to 20 days after the trip. WhatsApp photos get lost. Tally billing waits for the paper because clients reject invoices without it.

The money (the brief's own numbers)

Item	Working	Result
Annual freight	2,200 trips a month x Rs 32,000 x 12	Rs 84.48 crore

Working capital freed	84.48 crore x 14 days / 365 (POD-to-invoice 18 days down to 4)	Rs 3.24 crore
Interest saved	3.24 crore x 12%	Rs 38.9 lakh a year
Lost or disputed PODs	1.5% of PODs = 396 trips x Rs 16,000 (half the freight)	Rs 63.36 lakh a year
Total		about Rs 1.02 crore a year

What success means (the brief's targets)

- 80% of PODs in the system within 24 hours of delivery.
- POD-to-invoice from 18 days to 4 days.
- Lost or disputed PODs from 1.5% down to 0.5%.

The dashboard measures exactly these three, so the client can see on one screen whether the product is paying for itself. The targets and the baseline are settings, not hard-coded numbers (`TARGET_POD_WITHIN_24H_PCT`, `TARGET_POD_TO_INVOICE_DAYS`, `TARGET_LOST_PCT`).

Who uses it

User	What they need	What CargoOne gives them
Driver (basic phone, low literacy)	Take one photo and be done	One big button, icons, spoken Hindi / Marathi, no typing, works offline
Hired driver (no app)	The same, without installing anything	A private link by SMS, valid for one trip
Billing executive	Know which trips can be billed; handle doubtful PODs	Review queue with the photo and what the AI read; billing queue; invoices; Tally
Operations head	Book trips, assign trucks, chase missing PODs	Planning board, request inbox, live updates, Needs-attention list, live map
Managing Director	See whether the targets are met	Performance dashboard, working-capital figure, one-page digest
Client and receiver	Know where the load is; see the proof; pay	Client portal, shipment page, delivery code, online invoice payment

3. Scope: what I built and why

The brief's CORE BUILD lists one slice that must work end to end. I built that first and kept it solid, then added the surrounding workflow, because the optional "must have" items (SMS link, trip import, client POD link) and the real-life edge cases only make sense inside a complete trip. The brief lists live GPS tracking, trip planning and driver expenses as out of scope. I built light versions of each (section 11 and 12) as extra depth, labelled as such, and kept them from getting in the way of the core.

Layer	What is in it
Core slice (must work)	Driver POD capture, AI check with retake, review queue, billing queue with export, missing-POD dashboard
Rest of MUST HAVE	SMS link for hired drivers, Tally export, POD images on invoices, client link, trip import

Extra depth	Client request and quote flow, trip planning board, delivery code, expenses and advances, live updates, tracking with consent, WhatsApp / email notices, Razorpay invoice payment, privacy request register, AI benchmark screen

4. Architecture and why

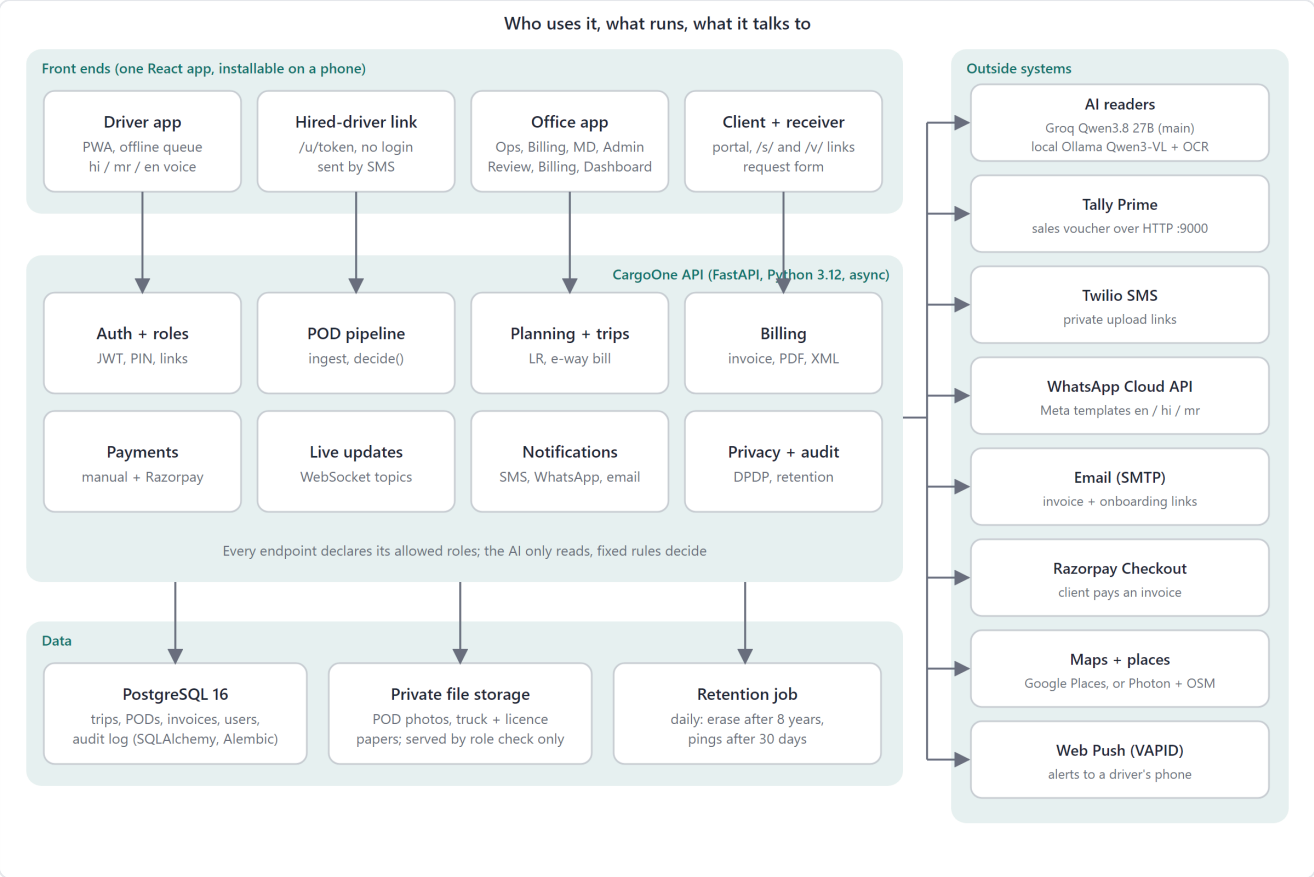


Figure 1. System architecture: front ends, API, data, and the outside systems it talks to.

The shape

One React web app (installable on a phone) talks to one FastAPI service over HTTPS and one WebSocket. The service owns a PostgreSQL database and a private file store. Everything outside the building (AI readers, Tally, SMS, WhatsApp, email, Razorpay, maps, push) is behind a small adapter, so each can be switched between a real provider and a simulated one by one setting.

Part	Choice	Why
Front end	Vite, React 19, TypeScript, Tailwind CSS v4, react-router	One codebase for the driver's phone, the office desk and the client's browser. Fast, typed, small build.
Phone app	PWA with service worker and IndexedDB	No app store, installs from a link, works offline. The brief's low-end Android drivers need that more than a native shell.
Back end	FastAPI (Python 3.12), async SQLAlchemy 2	Typed request validation by default (the brief wants validated inputs); async keeps AI and Tally calls from blocking.
Database	PostgreSQL 16, Alembic migrations (27 revisions)	Money, state and audit need transactions and constraints (for example the retake cap is also a database constraint).

AI	Groq Qwen3.8 27B (main), local Ollama Qwen3-VL 4B + glm-ocr / RapidOCR (fallback), Gemini and a scripted mock	Chosen by measurement, not preference (section 9). The local reader keeps the product working with no network and no data leaving the machine.
Live updates	In-process WebSocket hub	Screens refresh without reload. Only topic names travel on the socket; each screen refetches with its own login, so the socket can never leak data.
Maps	Leaflet on OpenStreetMap tiles; Photon (OpenStreetMap data) for place search; Google Places as an optional upgrade	Free by default; better search when the owner adds a key.
Tests and CI	pytest, vitest, Playwright, GitHub Actions	478 API tests, 49 web unit tests, 30 browser spec files; CI runs lint, migrations, API tests, web build and browser tests.

Decisions worth defending

- **The AI reads; rules decide.** The model returns fields. `api/app/pod/decision.py` is plain code that turns fields into verified, retake or review. It is testable, explainable to a client, and the same photo always gets the same decision. Swapping the model cannot loosen the rules.
- **The photo is the source of truth.** Stored bytes are never modified. When a person corrects a field, the AI's own reading is kept beside the correction.
- **Invoices are never sent automatically.** Making an invoice, pushing it to Tally and sending it to the client are three separate human clicks.
- **Offline first for the driver.** Capture saves to the phone first. Upload is idempotent, so a retry never creates a second POD.
- **Roles are enforced in the API,** not just hidden in the screens. Every endpoint declares its allowed roles; the tests check the 401 / 403 / 404 behaviour.
- **One process, one database.** A transporter of this size does not need microservices. The one known limit is that the live-update hub is in memory (section 18).

5. Roles and access

Six sign-in roles plus three kinds of person who never sign in. Admin passes every check. The matrix shows the day-to-day boundaries.

Area	Driver	Billing	Ops	MD	Client
Capture POD, loading photo, expenses, own messages	Yes (own trips)				
Review queue: decide (approve, retake, reject)		Yes	Yes	View only	
Reopen a rejected POD			Yes		
Billing queue, invoices, payments, Tally push		Yes	Yes		Own invoices
Cancel an unsent invoice			Yes		
Undo a recorded payment (admin only)					
Book and replan trips, assign truck and driver		View	Yes		
Requests inbox, quotes		View	Yes		Own requests
Setup: trucks, drivers, vendors, clients, branches		View	Yes		

Staff accounts (admin only)				
Performance dashboard		Yes	Yes	
Live map and driver location		Yes	Yes	Own shipment, rounded
Trip import	Yes	Yes		
AI benchmark screen	Yes	Yes	Yes	
Privacy requests (the officer role)			Yes	

Person without a login	How they get in	What they can see
Hired driver	Private link `/u/<token>`, SMS, one trip, 72 hours	That trip's destination and vehicle; the capture screen
Receiver (consignee)	Shipment link `/s/<token>`	Progress, timeline, map, their 4-digit delivery code. Never a price, an invoice or a phone number
Requester	`/request` form, then a signed status link	Where their request stands, the quote, a yes / no button

Drivers get a 404 (not 403) for trips that are not theirs, so a guessed ID reveals nothing. Passwords and PINs are Argon2 hashes. Five wrong sign-ins lock an account for 15 minutes. Link tokens are 256-bit random values; only their SHA-256 is stored.

6. Data model

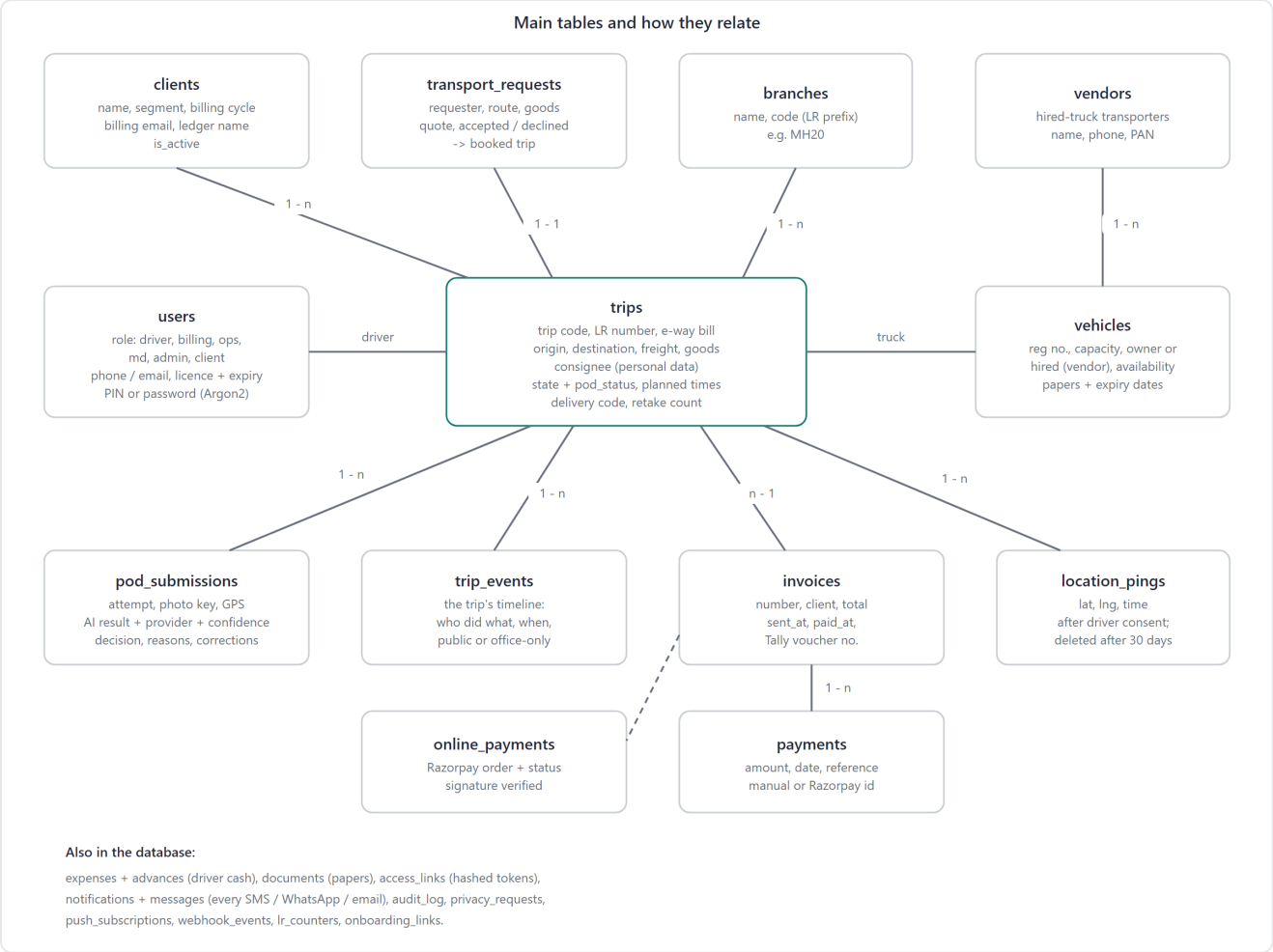


Figure 2. The main tables. The trip is the centre; everything else hangs off it.

Table	What it holds	Notes
trips	Trip code, LR number, e-way bill, route, freight, goods, consignee, state, POD status, planned times, delivery code	Consignee name, phone and email are personal data (DPDP) and are erased 8 years after the trip closes. One e-way bill can sit on only one live trip.
pod_submissions	One row per photo: storage key, GPS, capture time, what the AI read, which reader, confidence, decision, reasons, corrections	A DB constraint caps attempts; idempotency key makes retries safe.
trip_events	The trip's timeline with public or office-only visibility	Feeds the shipment page and the audit view.
invoices, payments, online_payments	Invoice header, each payment (manual or Razorpay), Razorpay orders	Part payments allowed; payment more than the balance is refused.
users	Drivers and staff with role, phone or email, licence number and expiry	Roles: driver, billing, ops, md, admin, client.
vehicles, vendors	Own and hired trucks with papers and expiry dates; the transporters that supply hired ones	A truck on a trip is not offered as available.
clients, transport_requests	Shippers and their incoming requests with quote state	A request becomes a trip; cancelling the trip reopens the request.

location_pings	Driver position after consent, while a trip is on the road	Deleted after 30 days.
expenses, advances	Driver cash out and cash in, with photos and approval	Gives the driver a running balance.
documents, access_links	Truck and licence papers in private storage; hashed link tokens	Replaced files are deleted.
notifications, messages	Every SMS, WhatsApp and email attempt with its status	Phone numbers masked; link secrets never logged.
audit_log, privacy_requests	Who did what; the DPDP request register	Photo views, review decisions, invoices and exports are all audited.
lr_counters	Per branch code and financial year serial for LR numbers	Numbers issued with a single insert-or-increment, so two people can never get the same one.

LR numbers follow `LR/<branch code>/<financial year>/<serial>`, for example `LR/MH20/26-27/900019`, and restart each April. A gap is normal (a cancelled trip never gives its number back); a duplicate is impossible.

7. The main flows

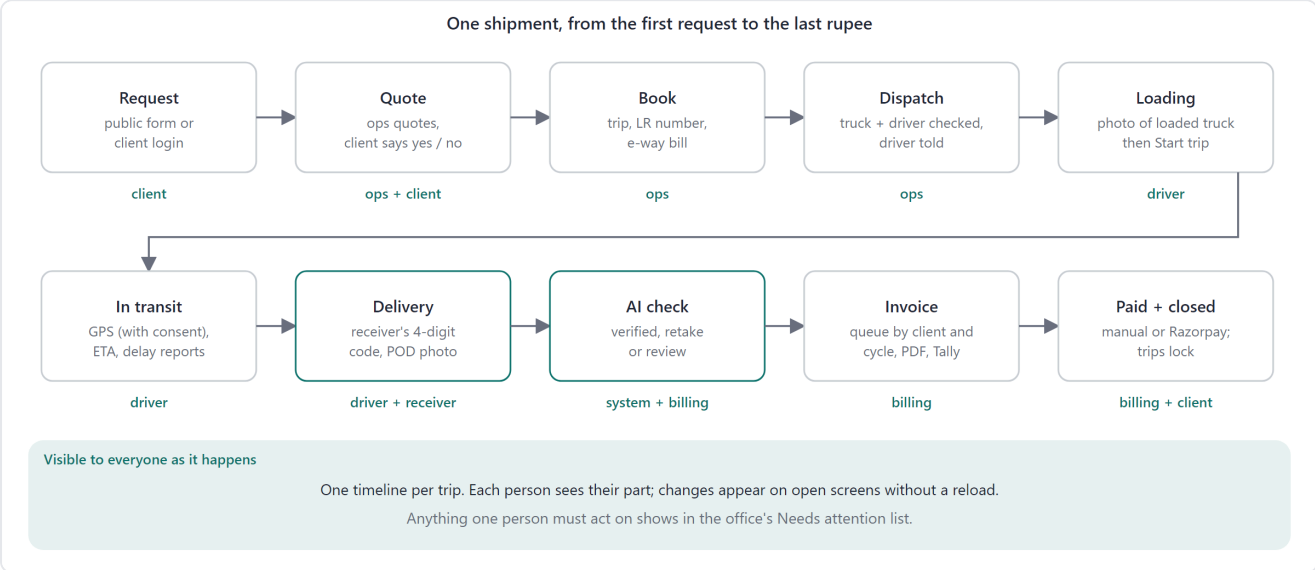


Figure 3. One shipment from request to paid invoice, and who acts at each step.

7.1 Order to booking

1. A new company fills the public `/request` form (consent to be contacted is stored with a timestamp), or an existing client asks from their portal.
2. Ops accepts with an optional quote. The requester sees the quote on their request page and answers yes or no. A trip cannot be booked from a quoted request before a yes. A no is shown to ops with the reason.
3. Ops books the trip: LR number is issued, the e-way bill and its validity are entered (it must cover the trip and sit on one live trip only), consignee name, phone and email are required so the delivery code has somewhere to go.
4. Truck and driver are checked for papers, capacity and clashes (the same truck or driver cannot be on two overlapping trips). The driver is told in the app, by WhatsApp and by push alert.

7.2 On the road

- The driver taps **Truck is loaded** and photographs the loaded truck and LR. Only then does **Start trip** work, and only from 3 hours before the planned start.
- With the driver's consent, the phone sends its position. The shipment page shows progress, the timeline, the position rounded to about 100 m and an arrival estimate.
- Delay or breakdown is reported in a few taps (no typing). The client and receiver are told; if the estimate passes the e-way bill's validity, the timeline says so.

7.3 Delivery and POD

1. The driver taps **I have reached**. The receiver is told the truck is at the gate.
2. The receiver opens their link and reads out a 4-digit delivery code. The driver enters it (five wrong tries lock it for 15 minutes; only the office can deliver without it, with a reason).
3. The driver photographs the signed LR. The app adds a dark strip below the picture (never over the paper) with the date, the time in IST and the GPS position; the time is the server's clock.
4. The photo is compressed under 300 KB, queued, uploaded, and checked (section 8). The driver sees a spoken and written result in their language: green tick, amber retake, or sent for checking.

7.4 Hired driver by SMS

The office presses **Send link** on a trip. A one-trip, 72-hour link goes to the driver's phone by Twilio SMS. The page reuses the same capture engine as the app (offline queue, voice, retake) with no login and no install. The link is also always shown on screen, so the office can paste it into WhatsApp.

7.5 Billing, Tally and payment

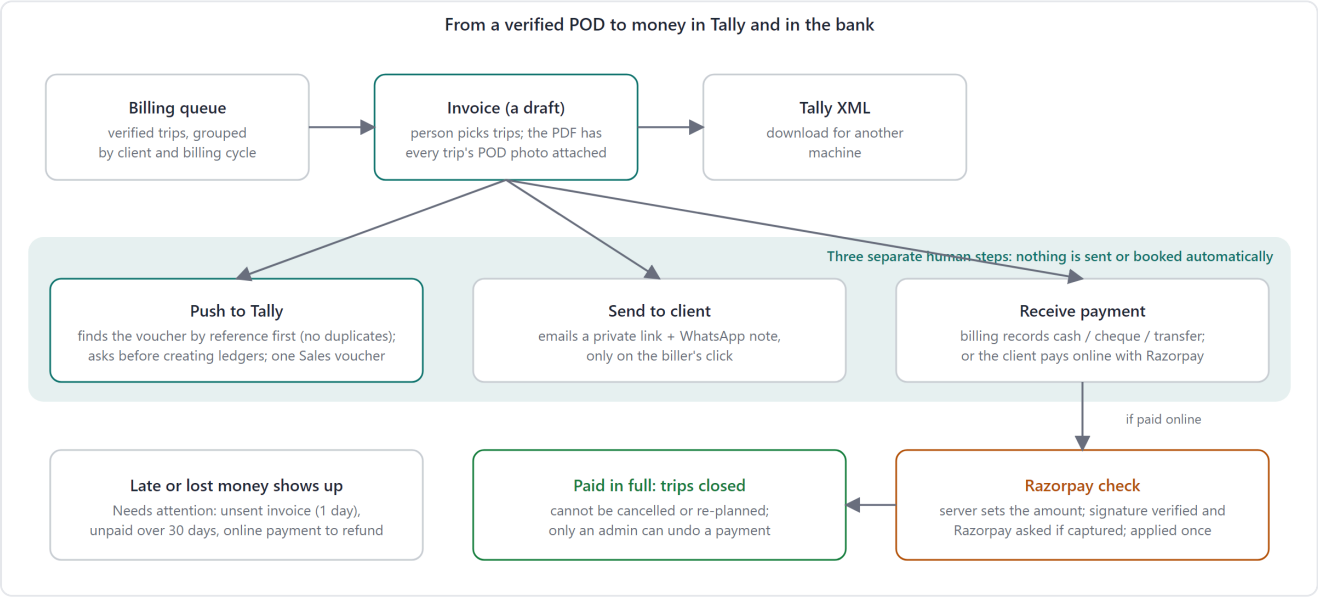


Figure 4. From a verified POD to Tally and the bank. Three human steps, no automatic sends.

- **Billing queue:** verified trips grouped by client and billing cycle (weekly, fortnightly, monthly). A person picks trips and creates a draft invoice.
- **PDF:** the invoice PDF attaches the POD photo of every trip on it, so the client cannot say there was no proof.
- **Tally:** see section 10.3.

- **Send to client:** emails a private link and a WhatsApp note, only when the biller clicks. **I sent it myself** records an invoice sent another way.
- **Payment:** billing records cash, cheque or transfer (amount, date, reference); or the client pays online with Razorpay (section 10.5). When payments cover the total the invoice is paid and its trips close; they can no longer be cancelled or re-planned.
- **Ways out of mistakes:** a rejected POD can be reopened with a reason; an unsent invoice can be cancelled and its trips go back to the queue; only an admin can undo a payment.

7.6 Live updates

Screens refresh without a reload. When a trip, review, invoice, request, expense or message changes, the API publishes only the topic name (for example "review") on a WebSocket. Each open screen refetches through its own authenticated call. Audience rules apply before anything is published: office sees all but messages, a driver sees only their own trips, expenses and messages, a client only its own trips, billing and requests.

7.7 Nothing falls between two people

The office's **Needs attention** list collects every step where one person acts and another must react: a new request, a client's no to a quote, a trip leaving within two days with no driver, a driver who has not opened the job, a breakdown, a late trip, shortage noted at the door, a POD missing or waiting for a person, an invoice made but not sent for a day, an invoice unpaid after 30 days, and any message that failed to send.

8. The AI part

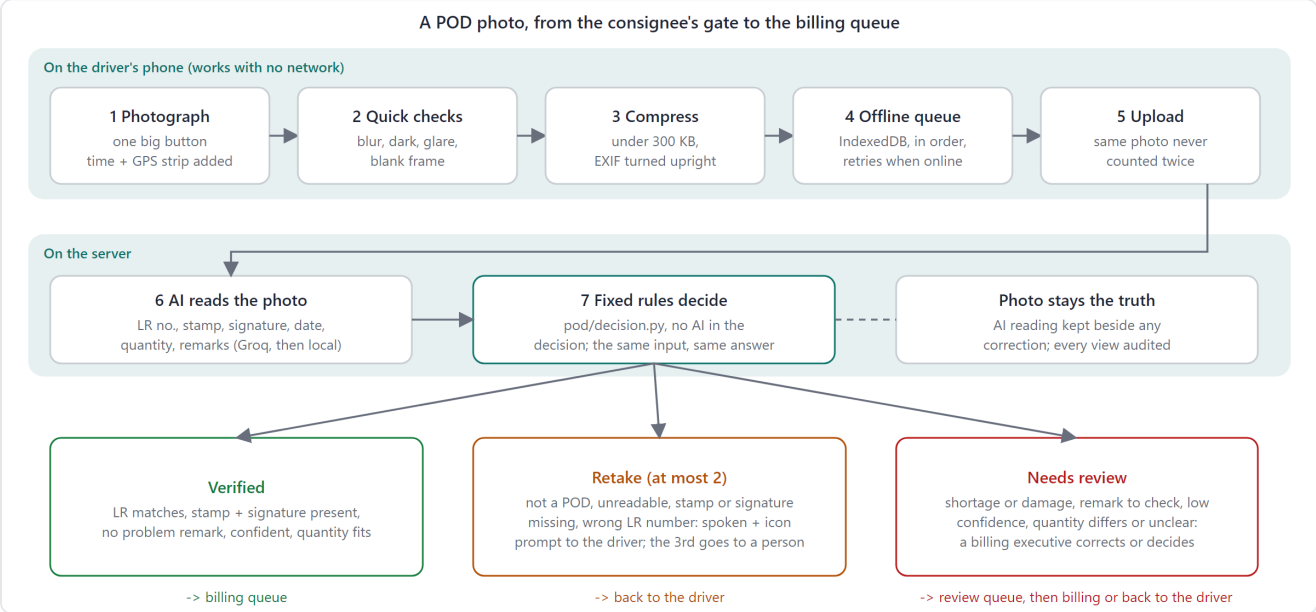


Figure 5. A POD photo from the gate to the billing queue.

8.1 What the AI reads

For every photo the model returns: is it a delivery paper, is it readable, the LR number, whether a consignee stamp is present, whether a signature is present, the delivery date, the quantity received (line by line when several goods) and the written total, the handwritten remarks, whether they mention shortage or damage, its confidence, and anything wrong it noticed. The prompt does not tell the model which LR number to expect (it could complete a half-visible number).

8.2 What the rules decide

Outcome	When	What happens
Retake	Not a POD; unreadable; stamp missing; signature missing; LR number differs from the trip's	The driver gets a spoken and written prompt with an icon. Capped at 2 retakes; after that a person decides.
Needs review	Shortage or damage remark; a remark that is not plainly fine; low confidence; quantity differs from the booking; several goods or units that cannot be reduced to one figure	A billing executive sees the photo beside what the AI read, can correct any field, approve, ask for a retake or reject.
Verified	Everything above is clear	Straight into the billing queue. A person can still open any verified POD later.

A missing date is not a reason to stop: the app records when the load was delivered. A quantity is compared with the booked total (weight with 5% tolerance, or package count). When nothing was booked to compare with, or nothing was read, nothing is flagged (an option, `POD_REQUIRE_QUANTITY`, sends unread quantities to a person).

8.3 What happens when the AI is wrong or unsure

Failure	Defence
Model misreads a handwritten shortage remark as harmless	Any remark that is not plainly fine ("received OK", "no shortage", a date) goes to a person (`remarks_to_check`). Caught on sheet S13 where Groq read a Marathi shortage remark as other words.
Model hallucinates or completes the LR number	The prompt never reveals the expected number, and the local reader accepts the number only when OCR sees it exactly. One digit off is another consignment.
Model is confident but wrong about the quantity	The quantity is read two ways (line by line and as quoted text) and ANY disagreement with the booking counts.
Cloud model is down or rate-limited	Falls back to the local reader; if both fail the POD goes to a person. A POD never silently disappears.
Model says verified but should not	The benchmark counts these as **unsafe errors** and the target is zero (section 9).
Driver keeps retaking	Capped at 2, enforced by the server and a database constraint; the 3rd goes to review.
Someone doubts the AI	The photo is untouched; the reviewer sees the AI's reading beside the photo; corrections are stored beside, never over, the AI's output.

8.4 Labels on the photo

In the review screen, what the AI found is drawn on the photo: boxes for the LR number, date, quantity and remarks (found by a local text reader) and for the stamp and signature (pointed out by the local vision model). Hovering a chip or a field highlights its box. The local reader draws these as it reads. For Groq, which returns only values, the app places the boxes once, the first time a POD is opened, and keeps them. The boxes are an aid for the reviewer and never change what the AI decided.

8.5 Providers and the privacy gate

Setting	Reader	Use
`AI_PROVIDER=groq`	Qwen3.8 27B on Groq (cloud)	Main reader, best on the benchmark

`AI_PROVIDER=local`	Qwen3-VL 4B + glm-ocr through Ollama, no cloud	Fallback; works with no network
`AI_PROVIDER=gemini`	Gemini 2.5 Flash	Works; the free quota is too small to run on
`AI_PROVIDER=mock`	Scripted answers	Default, all tests and demos

A photo may show names, signatures and phone numbers. Outside development, a cloud reader is used only when `CLOUD_AI_CONFIRMED=true`, which the owner sets after confirming a no-training paid tier. Until then the local reader works alone.

9. AI benchmark

I did not pick a model by reputation. I built a test set of real paper, ran every candidate on it, and chose on the measure that matters to the client: never let a bad POD through.

9.1 How the dataset was made

1. **22 sample LRs.** `docs/benchmark/lr_forms.pdf` has 22 pages, each the app's own printed LR (same layout as the PDF the app issues) with fake names and numbers. LR numbers start at 900001, far above anything the app will issue, so a test sheet cannot be confused with a real one.
2. **The receiver's part was left empty** (date, quantity received, remarks, stamp, signature). I filled each sheet by hand, in different inks and handwriting, with a real stamp where possible, to create one specific case per sheet: clean, no stamp, no signature, empty, wrong LR, shortage with remark, damage, short count with no remark, weighed loads, blank date, Hindi and Marathi remarks, faint or messy stamp, a harmless remark, and six multi-goods cases (totals that match, one short, with or without a remark, lines that do not add up, kilograms, mixed units).
3. **Photo conditions.** Sheet S01 was photographed seven ways: flat, angled about 30 degrees, dim, flash glare, crumpled with shadow, out of focus, and one corner cut off. 21 other sheets plus these 7 shots gives **28 photos**.
4. **An answer key.** `answers.csv` records, for every sheet, the expected stamp, signature, date, quantity text, line items, total, remark and the expected decision. The expected decisions are computed by the app's own rules, so the key moves with the rules. Sheets where a person could reasonably go either way (S14, S15, S22) are marked as judgement calls and accept more than one answer.
5. **A harness and a screen.** `app.devtools.benchmark` runs a model over the photos, saves a resumable JSON per run, and the office app's **AI benchmark** tab shows every run: pass rate, unsafe count, needless retakes, field accuracy, time, expected against actual for every photo, and the field-by-field comparison.

9.2 What is counted

Measure	Meaning	Target
Unsafe	A sheet that should NOT be verified (missing stamp or signature, wrong LR, shortage, different quantity) would have been verified	0, always first
Needless retake	A good sheet sent back to the driver	As few as possible
Safe but different	Sent to a person where a retake was expected, or the reverse	Acceptable
Fields right	How many individual fields were read correctly	Higher is better

9.3 Results (28 photos, 8 to 9 November 2026)

Run	Pass	Unsafe	Needless	Fields	Time / photo
Qwen3.8 27B on Groq, final rules (chosen)	**96.4% (27 of 28)**	**0**	1	97.9%	16 s
Qwen3.8 27B on Groq, before the final rules	89.3%	2	1	97.9%	16 s
Qwen3-VL 4B + glm-ocr (local, every field required)	78.6%	1	1	84.5%	34 s
Qwen3-VL 4B + glm-ocr, before the schema fix	85.7%	3	1	85.0%	32 s
Qwen3-VL 8B + glm-ocr (local)	67.9%	3	3	81.3%	57 s
Qwen3-VL 8B + glm-ocr, before the schema fix	71.4%	8	0	53.9%	53 s
Qwen3-VL 4B + RapidOCR (the first local reader)	71.4%	3	3	76.2%	12 s
MiniCPM-V 2.6 + glm-ocr (local)	21.4%	0	12	57.5%	50 s

The one non-pass in the chosen run is S15, a judgement-call sheet where the stamp was pressed over the signature, date and quantity.

9.4 What the benchmark taught me

- **glm-ocr was the biggest single win for the local reader.** It reads handwritten quantity, date and remarks almost perfectly. The first local reader (RapidOCR) only reads print.
- **A bigger local model was not better.** Qwen3-VL 8B did worse than the 4B on this paper and also needs more than a 6 GB graphics card has.
- **The benchmark found a real bug.** The local model's answer format required only six fields, so a larger model answered the bare minimum and left quantity, date and remarks empty. Every field is now required (they may still be null).
- **Both leaders still let a bad sheet through on their own.** Groq verified a cut-off corner it called readable (S01_cut) and misread the Marathi shortage remark (S13); the local pair took the consignor's and driver's signatures for the receiver's (S03). The final rules catch these without needing the reader to be right.
- **Two readers together would score better.** Judged from stored decisions: verify only when both readers say verify. That gives 26 of 28 with 0 unsafe and 0 needless retakes, because the readers fail on different sheets. It needs a cloud call and the local model for every POD, so I left it as a decision for the owner.

9.5 Honest limits

- **27 of 28 is tuned on this paper.** Two rules were adjusted after the first Groq runs (non-plain remarks go to a person; a cut corner is accepted when the LR is printed again at the foot). Treat the number as "tuned on this set", not a forecast for real client PODs. **Assumption A9:** real PODs will be worse until measured.
- **Printed sheets, my handwriting.** Real drivers, receivers, rubber stamps, rain-wet paper and truck-cab lighting are different. The next step is a larger set from the client's own PODs.
- **Gemini 2.5 Flash** was not ranked: the free daily quota ran out part way. **Llama 3.2 Vision** could not run on the installed Ollama. **PaddleOCR-VL** has no Ollama build.
- **Groq's free tier** allows about three photos a minute, so real traffic needs a paid plan; and a cloud model means the photo leaves the machine (see the privacy gate in 8.5).

10. Integrations

Each outside system sits behind an adapter and has two modes: **live** (credentials set) and **simulated** (default; nothing leaves the machine, the screen says so, and the app never pretends to send). The table is honest about what I could verify in the build period.

Integration	What it does in CargoOne	Mode in this submission	Setting names
Twilio SMS	Sends hired drivers their private upload link; reminders for a POD still owed	Code complete with a hard ceiling on real texts. **Simulated** here: India needs a registered sender and template (DLT), which is an owner task. The link is always shown on screen.	`SMS_PROVIDER`, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_FROM` or `TWILIO_MESSAGING_SERVICE_SID`, `SMS_MAX_TOTAL`
WhatsApp (Meta Cloud API)	Driver account link, new trip, trip change, request received and updated, shipment updates to client and receiver	Code complete with 6 templates x 3 languages (18 templates, utility category). **Recording mode** until Meta approves: every message is built and logged as "Not sent", with an Open WhatsApp button.	`WHATSAPP_PROVIDER`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_TEMPLATE_*`
Email (SMTP)	Invoice link to the client, one-time onboarding and reset links	Code complete; simulated until SMTP is set.	`EMAIL_PROVIDER`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURITY`, `SMTP_USER`, `SMTP_PASSWORD`, `EMAIL_FROM`
Tally Prime	One-click Sales voucher; XML download as the alternative	**Working** against a local Tally Prime (Connectivity on, port 9000). A hosted server cannot reach a Tally on someone's desk; that needs a connector, not built.	`TALLY_URL`, `TALLY_COMPANY`, `TALLY_SALES_LEDGER`
Razorpay Checkout	A client pays an invoice online with UPI, card or netbanking	Code complete and tested with mock verification. **Needs test keys** to run end to end. Not used for driver payouts (section 12.3).	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`
Groq, Ollama, Gemini	Read the POD photo	Groq is the main reader (needs a key); Ollama is the local fallback; Gemini works but is quota-limited.	`AI_PROVIDER`, `AI_FALLBACKS`, `GROQ_API_KEY`, `GROQ_MODEL`, `OLLAMA_URL`, `OLLAMA_MODEL`, `OCR_ENGINE`, `GEMINI_API_KEY`, `CLOUD_AI_CONFIRMED`
OpenStreetMap, Photon, Google Maps	Place search and exact pins on trips and requests; the live map; the driver's Navigate button	**OpenStreetMap + Photon work with no key.** Google Places, Geocoding and Maps JavaScript are an optional upgrade.	`PLACES_PROVIDER`, `GOOGLE_MAPS_SERVER_KEY`, `GOOGLE_MAPS_BROWSER_KEY`
Web Push (VAPID)	Alerts on a driver's phone: new trip, change, message	Code complete; live when both keys are set.	`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`

10.1 Twilio SMS

Trips, **Send link** creates a link for one trip and, when the driver's mobile is filled in, sends it. The wording is in `api/app/notify/sms.py` (`TEMPLATES`) and is what must be registered for DLT. Every real text is counted in the audit log and sending stops at `SMS_MAX_TOTAL`, so a trial balance cannot be spent by accident. If Twilio refuses,

the dialog shows its reason and the link is still good. If `SMS_PROVIDER=twilio` is set but a key is missing, the app stays simulated; it never pretends to send. The driver's mobile typed on that dialog is not stored; it is masked in the audit log.

10.2 WhatsApp

Messages go to drivers, requesters, clients and receivers in **their own language** (English, Hindi, Marathi). Meta requires one template per language, so each of the six notices exists three times (`name`, `name_hi`, `name_mr`). The texts are written as plain utility notices (no marketing words) so that Meta approves them as utility. **Setup > Messages** is the log of everything sent or recorded, with masked numbers and the reason for any failure.

10.3 Tally

- **Push to Tally** finds the company (the only open one, or `TALLY_COMPANY`) and looks first for a voucher with the same **Reference** (our invoice number). Tally does not refuse duplicates, so without this a second click would book it twice. A second click says already in Tally and adds nothing.
- If the client's ledger or the sales ledger is missing, the screen asks first, then creates the party under Sundry Debtors and the sales ledger under Sales Accounts. Nothing is created in Tally without that yes.
- It sends one Sales voucher: party debit, sales ledger credit, the invoice total, narration listing the LRs. The voucher date must fall inside the company's financial period; if not, the screen asks for another date. Tally's Educational mode accepts only the 1st, 2nd and last day of a month.
- The result is recorded on the invoice (**In Tally - voucher no.**) and in the audit log. Tally answers HTTP 200 even when it refuses; the real reason is read from the reply and shown.
- **Not built:** GST lines in the voucher, and a connector for a hosted server to reach Tally.

10.4 Maps, places and GPS

- **Place fields** (trip origin and destination, requests) suggest places while typing. Without keys they use Photon on OpenStreetMap data; with keys they use Google Places (New) and Geocoding. Picking a suggestion saves its exact latitude and longitude.
- **Pick on map** lets someone move a map under a fixed pin, search, or use their current location. The driver's **Navigate** button goes to that exact saved point.
- **The live map** is Leaflet on OpenStreetMap tiles. A hosted tile provider with a key is the production choice.
- Google's terms do not allow caching its search results, so Google answers are never cached here (the OpenStreetMap fallback is cached for a day). This is an open question for the owner if they expand (section 18).

10.5 Razorpay

When an invoice is sent and unpaid, the client sees **Pay Rs X online** on their login or private link. The server sets the amount from the invoice balance, never the browser. A payment counts only after the signature verifies and Razorpay's API confirms it is captured, for this order and amount. It is applied once, however many times it is reported (return, refresh, webhook). Money that arrives after the invoice was already settled is not added: it appears in Needs attention as an online payment to refund. Failed test payments change nothing. Refunds are made in the Razorpay dashboard and are not mirrored back yet.

11. GPS and tracking

The brief lists live GPS tracking as out of scope. The brief's core ask is the GPS stamp on the POD photo, which is built. I added consent-based tracking as extra depth, with privacy first.

Feature	How it works
GPS stamp on every photo	Position and server time are captured with the photo and printed in a strip below the paper. If GPS is slow or denied (4 s), capture never blocks.
Distance check	A delivery photo taken farther than `POD_MAX_DISTANCE_KM` (25) from the destination adds a tip for the reviewer and the driver.
Consent	The driver is asked as soon as there is a trip. They can say Not now; the choice lasts the day. Positions are collected only after they agree and only while a trip is on the road.
Movement states	Moving, stopped (over 150 m radius for 90 minutes) or no signal (nothing for 15 minutes), worked out by pure, tested rules in `tracking.py`.
Arrival estimate	Straight-line distance x 1.3 road factor / 40 km/h; tells the timeline when it passes the planned end or the e-way bill's validity.
Who sees it	Ops and MD (not billing) on the live map; every view of a path is audited. The client's shipment page shows the position rounded to about 100 m.
Retention	Position fixes are deleted after 30 days (`TRACKING_RETENTION_DAYS`).

12. The driver's side, and the money in it

12.1 The driver app

- **One screen, one trip, one big button.** The truck on the road first, then a delivery waiting for its photo, then the earliest trip to come. Four steps on the card: Loaded, On the road, Reached, Delivered.
- **Voice and icons.** Prompts are spoken in Hindi, Marathi or English and written on the result screen. Green tick means done, amber circular arrow means retake, an eye means sent for checking, cloud-off means saved on the phone. Language can be changed on the result screen (EN, hi, mr). Native-speaker review of the Hindi and Marathi wording was done by the owner.
- **Offline.** Loading photo, start, reached, delay and POD all save on the phone first and send when there is network, in order, without ever double-counting.
- **Sign-in.** Phone and a PIN, no typing of words. The first PIN is one-time and must be replaced.
- **Help.** One-tap Call the office (shown when `OFFICE_PHONE` is set), a Help tab, and messages from the office with a badge.

12.2 Expenses and advances

The driver records an expense with a photo; the office approves it; cash advances are recorded by the office. The driver's home screen shows a small **Cash with you** card: advances received, expenses approved, expenses waiting and the balance. It is hidden for a driver with no money activity. A private-link driver cannot see it.

12.3 What is not connected: driver payouts

Razorpay is connected only for a **client paying an invoice**. Paying a driver an advance or a reimbursement through Razorpay (payouts) is not built. Advances and approved expenses are recorded and balanced inside

CargoOne; the money moves outside it (cash or the company's bank). A payout integration needs Razorpay's payout product and KYC, and a decision on who approves transfers. I left it out by design (section 18).

13. UI and design decisions

The brief says good, minimal design counts as much as features: clean layouts, white space, a restrained palette, clear type, nothing on screen without a purpose. These are the decisions that follow from it.

Decision	Why
One accent colour on a neutral base; green, amber and red only for success, retake and problem	A restrained palette means the three status colours carry meaning instead of decoration. Dark theme is supported.
Driver screen has one primary action and no forms	Drivers read poorly and stand in a gate in the sun. A big button, an icon and a voice beat any field.
Every result is spoken, written and iconed	Never colour alone, never text alone.
Review screen: photo large on the left, what the AI read as editable fields on the right, keyboard shortcuts A / R / X	The photo is the source of truth, so it gets the room. A billing executive does this all day; shortcuts save clicks.
Boxes drawn on the photo, with a chip per finding that highlights on hover	A reviewer should check each finding against the paper in one glance.
Counts match everywhere	"Awaiting review" on Today equals the Review queue equals the dashboard. Missing and ready-to-bill numbers use the same rules on every screen.
Filters on every setup list with counts in the options	A fleet of hundreds needs to be narrowed (for example trucks by paper status: valid, due within 30 days, expired).
Loading, empty and error states on every screen	No blank screens, no dead buttons; the tests walk every page.
Phone-first layouts; desk layouts add columns	The same app serves the gate and the office.
Three languages in the interface (English, Hindi, Marathi) with a test that every key exists in all three	The brief names Hindi and Marathi; a missing translation fails the build.
Live updates instead of reload buttons	Ops should not have to refresh to see a POD arrive.

Key screens

Screen	For	What it shows
Driver home	Driver	One trip card, one big photo button, steps, cash card, call-the-office
Capture result	Driver	Green tick, retake or sent-for-checking, in voice and writing
Today (office home)	Ops, billing	Counts that open the right filtered list; Needs attention
Review list and detail	Billing, ops	Queue oldest first; photo with labelled boxes; editable fields; approve, retake, reject
Billing and invoices	Billing	Queue by client and cycle; invoices with Tally, send, payments, cancel
Performance	MD, ops	Three KPI tiles against target, branch league, weekly trend, missing-POD age buckets, working capital, one-page digest
Planning, trips, requests	Ops	Booking board with clash warnings, trip table with filters, request inbox with quotes

Setup	Ops, admin	Trucks, drivers, vendors, clients, branches, staff, message log
Live map	Ops, MD	Trucks on OpenStreetMap with moving, stopped and no-signal states
AI benchmark	Ops, billing, MD	Every model run, expected against actual per photo
Client portal and shipment page	Client, receiver	Shipments, timeline, proof, invoice, pay online, delivery code
Privacy notice and requests	Public, officer	Notice in three languages; request form; register and erasure

14. Security and privacy (DPDP Act 2023)

Area	What is in place
Access control	Role declared on every endpoint; drivers get 404 for other people's trips; link tokens are hashed 256-bit values scoped to one trip and one purpose; guessing is rate limited.
Secrets	None in the repository. `.env` is git-ignored; a non-development environment refuses to start with the default or a short `JWT_SECRET`.
Input	All request bodies validated by typed models; phone and e-way bill formats checked; the public request form has a hidden bot field, per-address and per-phone limits.
Images	Private storage only, served through role-checked endpoints with `Cache-Control: private, no-store`; type taken from the bytes, not the name; size and pixel limits; path-traversal guard.
No personal data in logs	Lists, dashboards, logs and AI error messages carry no consignee names or phones; numbers in the message log are masked.
Audit	Uploads, photo views, review decisions, invoices, exports, link creation and revocation, every Tally push.
Consent and notice	Public `/privacy` notice in English, Hindi and Marathi; consent stored with a timestamp on the request form; location only after the driver agrees.
Rights	A request form; an office register (admin and MD) with a lookup of what is held under a number, an erase action and a closing note; the office's own reply target is 30 days.
Retention	A daily job erases consignee name, phone and email, POD and loading photos, and the AI readings 8 years after a trip closes. The trip, LR, invoice and money stay for tax. Declined requests are removed after 180 days; location after 30.
Cross-border and cloud AI	A photo going to Groq or Gemini is a disclosure to a third party; the gate in 8.5 keeps it off until the owner confirms.
Safety rails	Demo seed and demo data refuse to run outside dev and test; tests use their own database and force the mock AI.

> This is an engineering record, not legal advice. Obligations under the DPDP Act should be confirmed with counsel. Open owner items are listed in `docs/16-dpdp.md`.

15. Testing and quality

Layer	What	Count

API tests (pytest)	Decision rules, AI response handling, access control, billing and payment rules, Tally XML, imports, privacy, links, live updates, dead-end recovery, location rules	478 tests
Web unit tests (vitest)	Compression ladder, offline queue, i18n coverage (every Hindi and Marathi key must exist), formatting	49 tests
Browser tests (Playwright)	Real Chromium including a phone-sized driver project: sign-in, every role's pages, offline capture, review, billing, requests, map picker, tracking, privacy, live update without reload	30 spec files
Lint and type checks	ruff on the API; TypeScript strict on the web	Run in CI
CI	GitHub Actions: API (lint, migrations, tests), web (type check, unit tests, build), end-to-end (Playwright against a throw-away database)	On every push

Most important logic gets the most tests: the decision rules, the quantity comparison, the LR number matching, the payment application, and the role gates. Tests are pinned to IST so dates do not drift with the machine's clock.

16. Sample data and how it is made

The brief's three sample trips (TRP-66120 verified, TRP-66134 retake, TRP-66141 needs review) are seeded exactly as given. Everything else in the demo is **fabricated and deterministic**, created by scripts. None of it is committed as data. No screen is labelled as demo and nothing on screen is typed in: every figure is calculated from the trips beneath it.

Script	What it makes
`python -m app.seed`	Branches, the brief's three trips, and the demo logins
`python -m app.seed_fleet`	8 own and 8 hired trucks, 6 hired-truck vendors, 12 drivers with licences, 5 clients, 9 staff; a few papers deliberately due or expired
`python -m app.demo_data`	About 140 trips over 75 days with rendered POD photos, invoices and payments. No truck or driver is on two trips at once
`python -m app.devtools.demo_video`	Three trips ready for recording the driver flow, matched to printed benchmark sheets

The data is shaped so the dashboard shows the client meeting its targets overall (80% of PODs within a day, 4 days to invoice, 0.5% lost) with one branch below target on purpose, so a red flag is visible. **This is simulated data to demonstrate the screens, not a result.** The product's real accuracy is the benchmark in section 9.

17. Assumptions

The brief says to use only the information in the problem statement and to label every assumption. These are the ones I made.

#	Assumption	Why, and the effect if it is wrong
A1	The brief's numbers (2,200 trips a month, Rs 32,000 average freight, 18 to 4 days, 12% interest, 1.5% lost) are taken as given and used for the dashboard's working-capital figure.	They are settings; changing them changes the dashboard without code.

A2	A trip has one booked total (a weight, a package count, or both). Several goods are typed as text and compared as one total.	A booking with a line per good would need a new table and a per-item check.
A3	Billing cycles are weekly, fortnightly or monthly per client.	The brief says "by client and cycle" without naming cycles.
A4	Branches are Chhatrapati Sambhajinagar (HQ), Pune and Nagpur, with LR prefix MH20 by default.	The brief mentions branches but names none. Demo data only.
A5	The LR format is `LR/<code>/<financial year>/<serial>` and restarts each April.	The brief shows LR numbers only as labels such as LR-88120.
A6	Hired drivers have a phone that receives SMS and opens a link; they do not install anything.	Matches the brief; a feature phone without a browser would not work.
A7	Receivers have a phone and an email so the delivery code can reach them.	Imports and old trips may lack them; the office can deliver without the code and must say why.
A8	Company A has Tally Prime with an HTTP connection on a machine the API can reach.	A hosted API cannot see a Tally on a desk; a small connector would be needed.
A9	Real client PODs will be harder than my hand-filled printed sheets.	Benchmark numbers are tuned on this set and are not a forecast (section 9.5).
A10	Quantity tolerance is 5% either way for weights; package counts must match exactly.	A setting in the rules; the client may want another number.
A11	A POD is "missing" once a delivered trip is more than a day old with no usable POD.	Drives the Today count, the filtered list and the dashboard; the brief's 80% within 24 h target uses the same day.
A12	Photos and consignee data may be kept 8 years after a trip closes, then erased.	Chosen to match tax record keeping; counsel should confirm.
A13	GST is not in the Tally voucher (party debit, sales credit only).	The brief does not mention GST; adding it is a known next step.
A14	Hindi and Marathi voice prompts use the phone's text-to-speech, with the wording reviewed by the owner.	A recorded-voice option is a next step if a low-end phone has no Hindi voice.
A15	Names, companies, phone numbers and e-way bill numbers in the demo are invented. Only the three sample trips come from the brief.	No real personal data is in the repository.
A16	The Razorpay integration is for clients paying invoices, not for paying drivers.	The brief asks for no payments; "mock payments" are allowed, and this is the useful one.

18. What I cut and what I would do next

18.1 Cut, and why

Cut	Why
Driver payouts through Razorpay	Needs Razorpay's payout product, KYC and an approval policy. Advances and expenses are recorded and balanced inside CargoOne instead.
A live chat between ops and clients	Requests, quotes and the shipment timeline already cover what a client needs to ask. A chat adds a channel nobody owns.
Per-good line items on a booking	One booked total keeps the quantity check simple and explainable (A2).

GST on Tally vouchers	Not in the brief (A13).
A Tally connector for a hosted server	A local Tally works; hosting needs a small agent next to Tally (A8).
Real SMS and WhatsApp sending	Both are code complete; each needs the owner's registered sender and Meta approval, which I cannot do for them.
Recorded Hindi and Marathi audio	The phone's text-to-speech is used (A14).

18.2 Known limits

- **Live updates run in one process.** With several API instances, a message published in one would not reach a screen connected to another. Redis (for example Upstash) or Postgres LISTEN / NOTIFY is the fix; the topic-name design makes it a small change.
- **The rate limiter is in memory** for the same reason.
- **Not deployed.** The product runs locally and in CI. `deployment.md` describes a deployment shape but I have not run it on a hosted server, so there is no live link.
- **The AI is measured only on printed sheets** with my own handwriting (A9).
- **Google's terms on stored coordinates** (a pin's latitude and longitude beside each trip) should be confirmed if the product expands; the alternative is to store the place id only.
- **Real phones.** The offline queue and compression are tested in a phone-sized browser; a real low-end Android pass is still to do.

18.3 Next, in order

1. Measure on 100 to 200 of the client's own PODs and tune on those, including a decision on two readers together.
2. Deploy: Docker images, a managed Postgres, object storage for photos, HTTPS, and Redis for live updates.
3. Register the SMS sender (DLT) and get the WhatsApp templates approved; switch both to live.
4. A Tally connector so a hosted server can push to Tally on the client's desk; add GST lines.
5. Per-good line items and a per-item quantity check.
6. Driver payouts if the client wants them, with an approval step.
7. Real-phone testing on the oldest Android handsets the drivers carry.

Appendix A. Feature checklist

Status: **Built** = works and is tested. **Built, needs credentials** = code complete; simulated until keys are set. **Not built** = deliberately left out.

Feature	Status
Driver app: one big photo button, icons, voice prompts hi / mr / en, PWA install	Built
Photo stamped with GPS and server time; compression under 300 KB; quick blur / dark / glare checks	Built
Offline capture queue, idempotent upload, retry	Built
Hired-driver private SMS link (one trip, 72 hours)	Built, needs credentials (Twilio)
AI POD check: LR, stamp, signature, date, quantity, remarks; retake capped at 2	Built

Groq main reader, local Ollama fallback, Gemini, mock; cloud gate	Built
Review queue with labelled boxes on the photo, correction beside the AI output	Built
Reopen a rejected POD; cancel an unsent invoice	Built
Billing queue by client and cycle; invoice PDF with POD photos; Tally XML	Built
Push to Tally Prime (duplicate check, ledger creation on yes)	Built (needs Tally running)
Send invoice to client by email and WhatsApp (human click only)	Built, needs credentials
Payments: manual record, part payments, undo by admin; trips close when paid	Built
Razorpay online invoice payment with signature and server verification	Built, needs credentials
Razorpay driver payouts	Not built
Performance dashboard: no-POD trips, POD-to-invoice by branch, 24-hour rate, lost rate, working capital, weekly trend, digest	Built
Client POD link and client portal; receiver shipment page with delivery code	Built
Daily trip import (CSV or Excel) with LR and e-way bill, preview then confirm	Built
Client request form, quote, accept or decline, client login by one-time link	Built
Trip planning board with truck, driver and e-way bill clash checks	Built
LR number generator and LR document (PDF)	Built
Driver GPS tracking with consent, movement states, ETA, live map	Built
Place search and map picker: OpenStreetMap and Photon by default, Google optional	Built
Expenses, advances, Cash-with-you card	Built
Messages between office and drivers; Web Push alerts	Built (push needs keys)
WhatsApp notices in three languages (18 templates)	Built, needs credentials
Live updates over WebSocket, no reload	Built
Needs-attention list across the whole shipment	Built
Setup: trucks (papers and expiry), drivers (licences), vendors, clients, branches, staff	Built
Privacy: notice, request register, erasure, 8-year retention job, grievance contact	Built
Audit log; role-checked endpoints; hashed link tokens; rate limits	Built
AI benchmark screen and harness; 28-photo dataset with answer key	Built
English, Hindi, Marathi interface; dark theme	Built
Progress page showing build status	Built
Live chat between ops and clients	Not built
Per-good line items; GST in Tally; hosted Tally connector	Not built

Where to find things in the repository

Path	What is there
`api/app/pod/decision.py`	The rules that turn what the AI read into verified, retake or review

`api/app/ai/`	AI providers, the registry and the box locator
`api/app/routers/`	One file per area; each endpoint declares its roles
`api/app/billing/`	Billing periods, invoice PDF, Tally XML and the Tally push
`api/app/notify/`	SMS, WhatsApp, email and push
`api/migrations/`	Alembic revisions
`web/src/pages/`	Every screen
`web/src/lib/`	Offline queue, compression, live updates, i18n
`docs/`	Setup guides for each integration, the decision log, the roadmap, the benchmark and the DPDP record